

ReCarga — Dashboard de Recarga Veicular

Sprint 3 — Prototipagem Funcional e Integração · GoodWe Challenge · FIAP

Equipe envolvida

Nome	RM
Olavo Dadario Vianna Barreto	569272
Mateus de Oliveira Fernandes Neves	572431
Angela Sousa Takezawa	570797
Paulo Henrique Lira Bilac de Araujo	569496
Pedro Soares de Souza	571285
Jhon Cutile Titirico	571976

Sobre o projeto

O **ReCarga** é um protótipo de sistema inteligente para gerenciamento de recarga de veículos elétricos, desenvolvido para o GoodWe Challenge. A solução simula um estacionamento com múltiplas vagas de recarga que compartilham uma capacidade elétrica limitada, gerenciada automaticamente para evitar sobrecargas e priorizar diferentes planos de cobrança (pré-pago e pós-pago).

Na Sprint 3, o protótipo evoluiu para demonstrar de forma explícita a **sinergia entre energia renovável, automação e as tecnologias escolhidas**: o servidor passou a simular a geração de um painel solar local ao longo do dia e a somar essa geração à capacidade da rede elétrica, de modo que o próprio motor de automação (o algoritmo que distribui a energia entre as vagas) decide com base na disponibilidade solar em tempo real — e não apenas com base na rede.

Esquema detalhado de integração dos componentes

O protótipo atual é **simulado em software**, mas foi desenhado para espelhar fielmente a arquitetura física planejada nas Sprints 1 e 2 (sensor de efeito Hall + ESP32 + QR Code), agora acrescida do módulo de geração solar. O diagrama de blocos abaixo mostra como cada componente se conecta e atua em conjunto, da geração de energia e leitura de corrente até a exibição no dashboard e o retorno de comandos do usuário/gestor:

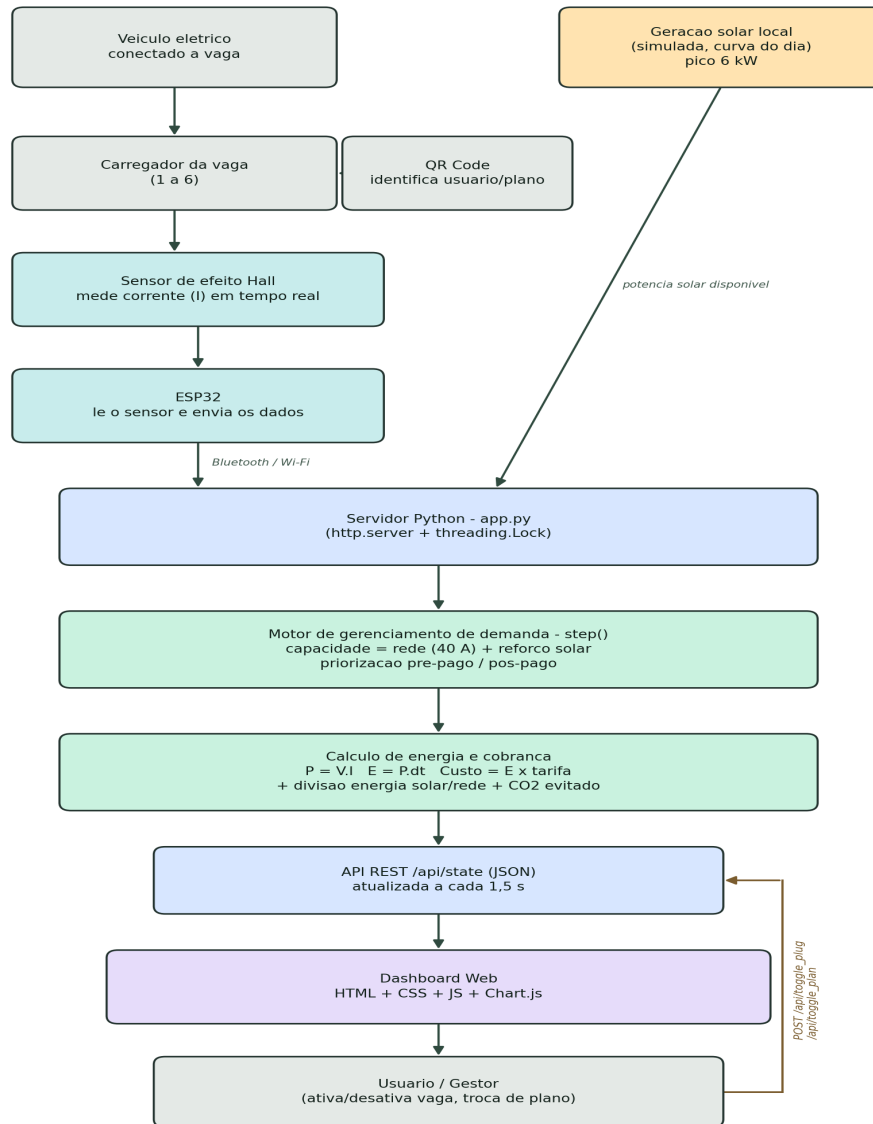


Figura 1 — Diagrama de blocos da integração de componentes do ReCarga (Sprint 3).

Fluxo resumido

- O veículo é conectado à vaga e identificado (protótipo físico: via **QR Code**; protótipo simulado: via seleção manual na interface).
- O **sensor de efeito Hall** mede a corrente instantânea consumida pelo carregador.
- O **ESP32** lê o sensor e transmite o valor via **Bluetooth/Wi-Fi** para o servidor.
- Em paralelo, o módulo de **geração solar simulada** (`solar_generation_kw()`) calcula a potência solar disponível naquele instante, seguindo uma curva de dia (nascer → pico ao meio-dia → pôr do sol).

- O servidor Python (app.py) executa o **motor de gerenciamento de demanda** (step()), somando a geração solar (convertida em corrente equivalente) à capacidade fixa da rede (40 A) e redistribuindo essa capacidade dinâmica entre pré-pago e pós-pago.
- A partir da corrente, o sistema calcula **potência, energia acumulada, custo, a divisão entre energia solar e de rede**, e o **CO₂ evitado** pelo uso da energia solar.
- Esses dados são expostos pela **API REST /api/state** em formato JSON, atualizada a cada 1,5 s.
- O **dashboard web** consome essa API e atualiza gráficos, indicadores e o card de geração solar em tempo real.
- O usuário/gestor interage de volta com o sistema (ligar/desligar vaga, trocar plano), fechando o ciclo de automação.

Justificativa técnica das escolhas

Escolha	Justificativa
Sensor de efeito Hall	Mede corrente sem inserir resistência em série (medição não invasiva), de baixo custo e compatível com o ADC de microcontroladores.
ESP32	Wi-Fi e Bluetooth integrados, múltiplas entradas analógicas e baixo custo, permitindo ler vários sensores e transmitir dados sem hardware adicional de comunicação.
QR Code por vaga	Solução simples e de baixo custo para identificar usuário/veículo e plano de recarga, sem exigir autenticação complexa nesta fase do protótipo.
Python + http.server (biblioteca padrão)	Evita dependências externas (como Flask), simplificando a execução do protótipo em qualquer máquina com Python instalado.
threading.Lock e ThreadingHTTPServer	Garantem leitura/escrita segura do estado compartilhado mesmo com requisições simultâneas, evitando condições de corrida.
API REST em JSON (/api/state)	Desacopla backend (simulação) de frontend (dashboard), permitindo futuramente plugar dados de hardware real sem alterar a interface.
Chart.js	Biblioteca leve, sem necessidade de build, suficiente para exibir a evolução da receita e do consumo em tempo real.
Priorização pré-pago/pós-pago	Reproduz um cenário comercial realista: garante capacidade para quem já reservou energia e usa a capacidade excedente para os demais.
Módulo de geração solar simulada (solar_generation_kw)	Modela a curva de um painel solar ao longo do dia e soma essa potência à capacidade da rede. É essa integração que conecta explicitamente energia renovável e automação: o motor de demanda decide com base em uma capacidade que varia conforme a disponibilidade solar.

Resultados e dados funcionais apresentados

O protótipo gera continuamente dados funcionais, atualizados a cada 1,5 segundos pelo motor de simulação (`step()` em `app.py`):

- **Corrente por carregador (A):** a partir da corrente-alvo do plano, com fator de escala de prioridade e jitter de $\pm 10\%$ simulando oscilação real.
- **Potência instantânea (kW):** $P = V \times I$, tensão fixa simulada de 230 V.
- **Energia acumulada (kWh):** $E = P \times dt$ (dt = intervalo de tempo), integrada a cada ciclo.
- **Custo por sessão (R\$):** $\text{Custo} = E \times \text{tarifa}$ (R\$ 1,35/kWh pré-pago; R\$ 0,98/kWh pós-pago).
- **Carga total do barramento (A):** soma da corrente de todas as vagas ativas, comparada à capacidade dinâmica (rede 40 A + reforço solar do instante).
- **Eventos de sobrecarga evitados:** contador de vezes em que a demanda excedeu a capacidade restante e precisou ser limitada/enfileirada.
- **Receita acumulada (R\$):** soma do custo de todas as vagas, por plano, em série temporal.
- **Status de cada vaga:** carregando, reservado, limitado, em fila ou livre.
- **Potência solar disponível (kW):** curva de dia simulado (0 kW à noite, pico de 6 kW ao meio-dia).
- **Reforço de capacidade vindo do sol (A):** ampères extras disponíveis no barramento graças à geração solar.
- **Energia solar × energia de rede (kWh):** divisão, a cada ciclo, do consumo entre geração solar e rede.
- **CO₂ evitado (kg, estimado):** energia solar consumida × fator médio de emissão da rede (valor ilustrativo).
- **Sobrecargas evitadas graças ao reforço solar:** quantas vezes o sol permitiu atender a demanda sem limitar/enfileirar.

Esses dados podem ser conferidos ao vivo em `http://localhost:5000` e são os mesmos exibidos no vídeo de demonstração da Sprint 3.

Conexão com os conteúdos da disciplina

- **Sinergia entre energia renovável e automação:** o motor de demanda soma a geração solar simulada à capacidade da rede a cada ciclo e redistribui automaticamente essa capacidade entre pré-pago e pós-pago — conectando diretamente os dois pilares pedidos no enunciado.
- **Eficiência energética e automação inteligente:** o algoritmo atua como uma malha de controle automática, redistribuindo a corrente sem intervenção humana, respeitando o limite físico do barramento — o mesmo princípio de sistemas de gerenciamento de energia (EMS) reais.
- **Sustentabilidade e energia renovável:** ao reforçar a capacidade disponível com energia solar, o sistema reduz o consumo de energia não-renovável e evita emissões, quantificado pelos indicadores de energia solar acumulada e CO₂ evitado.
- **Sistemas embarcados e IoT:** a arquitetura (sensor de efeito Hall → ESP32 → comunicação sem fio → servidor) segue o modelo clássico de um sistema IoT de monitoramento de energia.
- **Modelagem matemática aplicada:** fórmulas de potência, energia, custo e a curva de geração solar ($P_{\text{solar}} = P_{\text{pico}} \cdot \sin(\pi \cdot \text{fração_do_dia})$) exibidas em tempo real conectam os conceitos elétricos da disciplina à prática do protótipo.

Repositório GitHub: [inserir link do repositório] | **Vídeo de demonstração (YouTube, não listado):** [inserir link do vídeo]